

FinGuard AI Video Guideline

FinGuard AI Video Guideline

Purpose

This document provides a structured video script for the FinGuard AI project demo. It is written for three group members and covers:

- product description and problem solved
- system architecture and backend components
- fraud detection logic and performance
- testing instructions and limitations
- team member contributions

Video Structure

1. Opening and Product Description (Member 1)

****Member 1 script:****

"Hello, we are presenting FinGuard AI. This system is a financial advisory backend that helps users detect suspicious transactions."

In many personal finance situations, people can make risky transfers or take loans without understanding the consequences. FinGuard AI helps users understand the risks and make informed decisions.

We chose an expert system style for this project because financial guidance must be explainable. Instead of using a black-box model, we use a rule-based system that can provide clear reasoning for its decisions.

2. Architecture and Technologies (Member 2)

****Member 2 script:****

"The backend is built in Python using FastAPI. The system is divided into several layers:

- the API layer, which receives requests;
- the authentication layer, which secures access with bearer tokens;
- the business logic layer, which handles fraud scoring and credit advice;
- the data layer, which stores users, sessions, transactions, and alerts in SQLite.

The main API routes are `/register`, `/login`, `/transactions`, `/credit`, `/dashboard`, and `/alerts`. We chose FastAPI because it is fast and easy to use.

For login, the backend verifies passwords securely and generates a session token. All protected actions require `Authorization: Bearer <token>` header.

3. Fraud Detection and Performance (Member 3)

****Member 3 script:****

"FinGuard AI detects fraud using a rule-based scoring algorithm. Every transaction starts with a base score, and the system adds points for suspicious behavior:

- large amount if the transaction is above 1000;
- transaction size relative to monthly income;
- transaction size relative to monthly expenses;
- unusual location outside `'home'`, `'office'`, or `'local'`.

The final score is converted into a risk level of low, medium, or high. If the score is medium or high, the backend creates an alert and notifies the user via email or SMS.

This approach is strong for a prototype because it is deterministic and easy to explain. The main limitation is that it is not a production-ready system, as it lacks real-time data and advanced machine learning models.

4. Demo Instructions (Member 1)

****Member 1 script:****

"For the live demo, we will show these steps:

1. Register a new user with basic financial details.
2. Log in and receive the bearer token.
3. Submit a transaction through the `/transactions` endpoint.
4. View the dashboard and alerts to confirm the system shows the risk and recommendation.

The README includes exact commands for running the backend and using the API. We also provide a test client script that automates the demo.

5. Performance Summary and Limitations (Member 2)

****Member 2 script:****

"The system responds quickly under normal use and handles every request in a few milliseconds on a local machine. It is designed for educational purposes.

Key limitations are:

- the fraud logic is heuristic and rule-based;
- SQLite is used for ease of setup, not for high-load production;
- there is no full frontend in this repository, only the backend API.

Despite these limits, the design is valuable for learning because it clearly shows the flow from user input to decision output and monitoring.

6. Member Contributions (Member 3)

****Member 3 script:****

"Our team contributions are:

- Member 1: designed the product concept and implemented user registration and authentication.
- Member 2: developed the fraud scoring logic, credit advice rules, and alert system.
- Member 3: wrote the project documentation, README, and the final demonstration guideline.

We worked together to test the backend and ensure the API endpoints function correctly."

7. Closing (All Members)

****Closing script:****

"Thank you for watching our FinGuard AI demo. We hope this shows how the system can help users make safer financial decisions."

Notes for Recording

- Keep the language natural and avoid jargon.
- Use short sentences and explain each step clearly.
- Show the API commands and the response values when demonstrating the demo.
- Highlight the transparency of the scoring rules and why the expert system approach is useful.